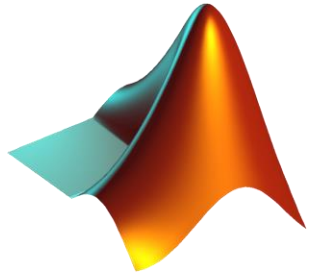
Politecnico di Torino

ROS Basics with MATLAB



P. David Cen Cheng
pangcheng.cencheng@polito.it
Andrea Usai
andea.usai@polito.it

Robotics 2023/2024

ROS Topics vs Services vs Actions

Topics

- Should be used for continuous data streams (sensor data, robot state, etc).

- Are for continuous data flow. Data might be published and subscribed at any time independent of any senders/receivers. Many to many connection. Callbacks receive data once it is available. The publisher decides when data is sent.

ROS Topics vs Services vs Actions

Services

- Should be used for remote procedure calls that terminate quickly, e.g., for querying the state of a node or doing a quick calculation such as IK (inverse kinematics).
  - They should never be used for longer running processes, in particular processes that might be required to preempt if exceptional situations occur and they should never change or depend on state to avoid unwanted side effects for other nodes.

- Simple blocking call. Mostly used for comparably fast tasks as requesting specific data. Semantically for processing requests.

ROS Topics vs Services vs Actions

Actions

- Should be used for any discrete behavior that moves a robot or that runs for a longer time but provides feedback during execution.
- The most important property of actions is that they can be preempted and preemption should always be implemented cleanly by action servers.
- Actions can keep state for the lifetime of a goal, i.e., if executing two action goals in parallel on the same server, for each client a separate state instance can be kept since the goal is uniquely identified by its ID.

ROS Topics vs Services vs Actions

Actions

- Slow perception routines which take several seconds to terminate or initiating a lower-level control mode are good use cases for actions.
- More complex non-blocking background processing. Used for longer tasks like execution of robot actions. Semantically for real-world actions.

Note: preemption is the act of temporarily interrupting an executing task, with the intention of resuming it at a later time. This interrupt is done by an external scheduler with no assistance or cooperation from the task.

ROS Going beyond

- Automated Parking Valet with ROS in MATLAB

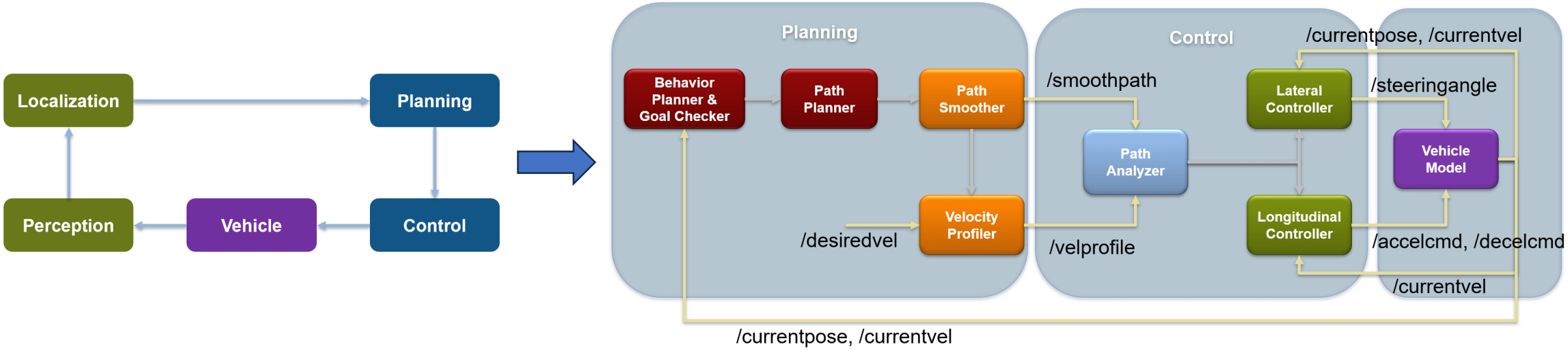
This example puts together, in a simple way, the ingredients needed for an automated parking using MATLAB: >> openExample('ros/AutomatedParkingValetWithROSInMATLABExample') *

The vehicle's automated systems are expected to take over control and steer the vehicle to an available parking spot. Such a function makes use of multiple on-board sensors. For example:

- Front and side cameras for detecting lane markings, road signs (stop signs, exit markings, etc.), other vehicles, and pedestrians
- Lidar and ultrasound sensors for detecting obstacles and calculating accurate distance measurements
- Ultrasound sensors for obstacle detection
- IMU and wheel encoders for dead reckoning

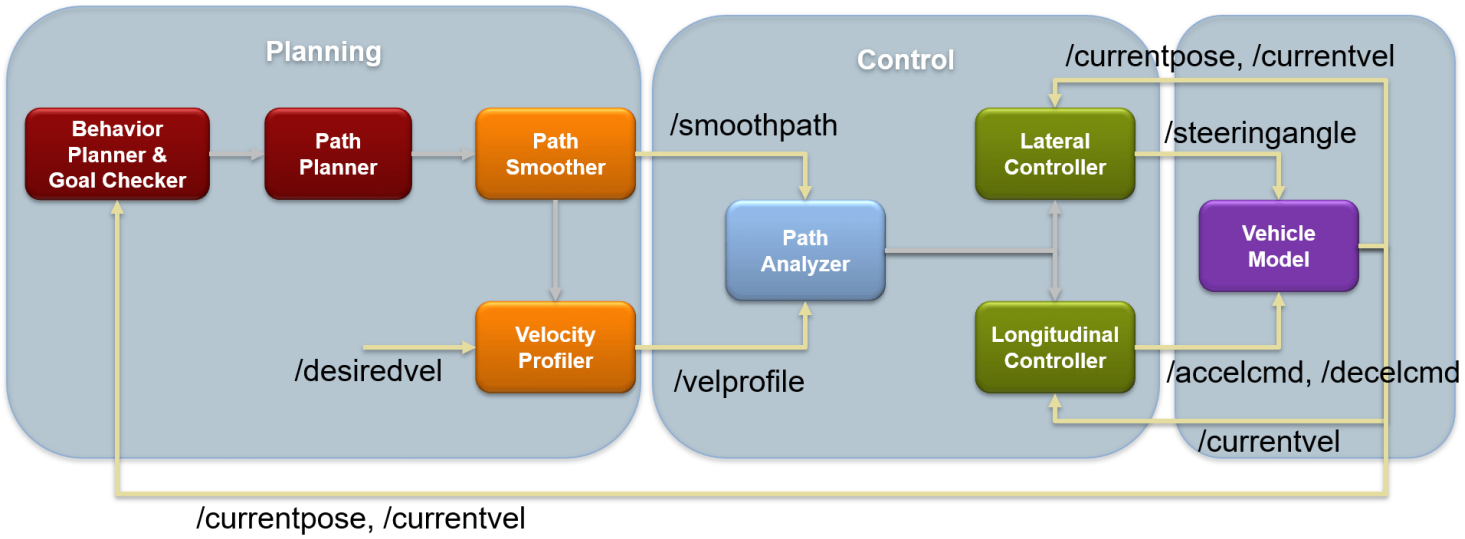
*Example included in “Automated driving toolbox”

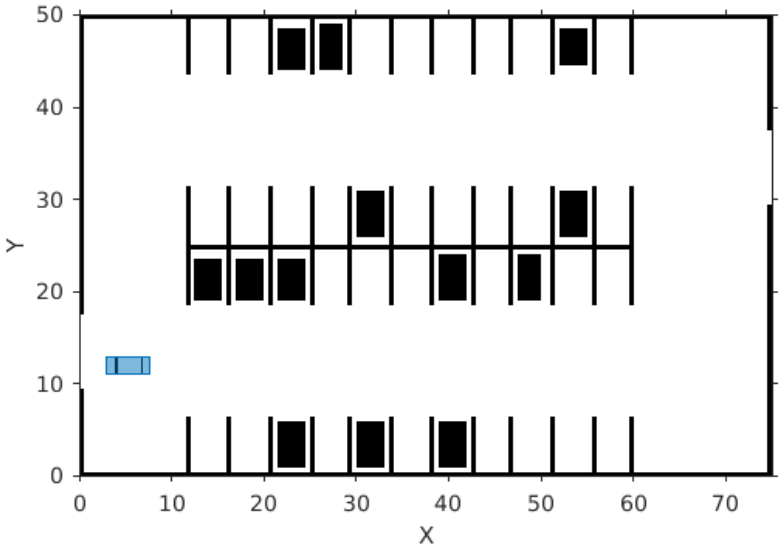
Automated Parking Valet with ROS in MATLAB



For simplicity, this example uses prerecorded data to substitute localization information

Automated Parking Valet with ROS in MATLAB

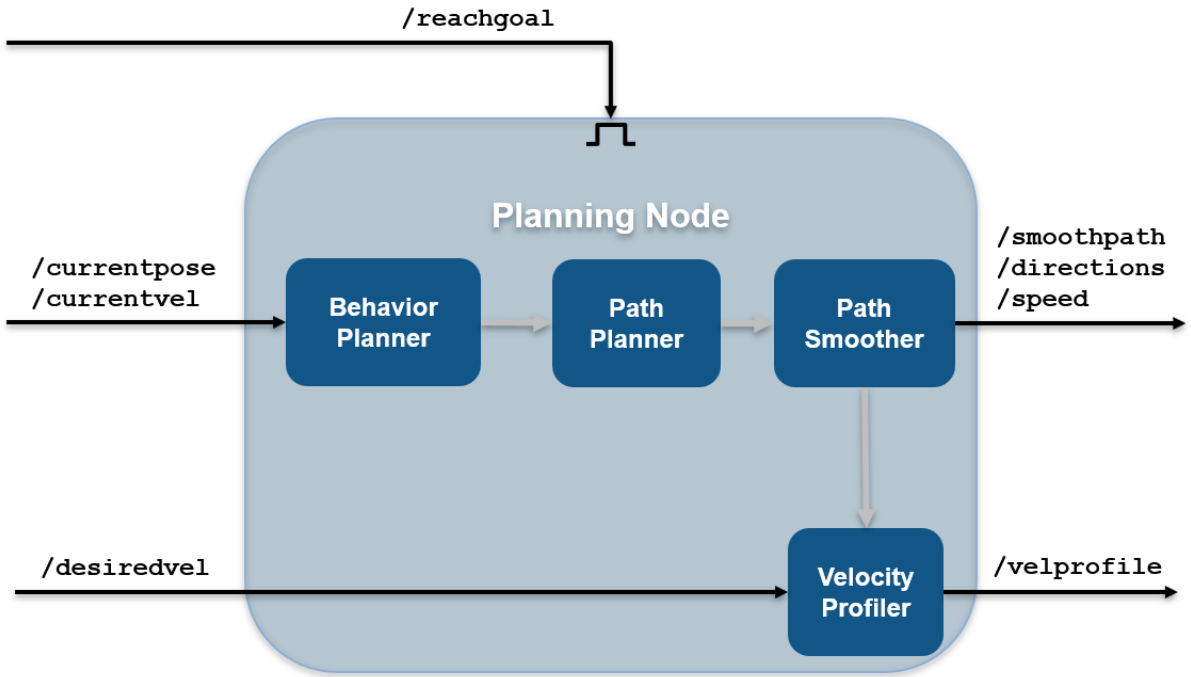




For simplicity, this example uses prerecorded data to substitute localization information

Automated Parking Valet with ROS in MATLAB

- Planning node:



Publishes to these topics

- /smoothpath
- /velprofile
- /directions
- /speed
- /nextgoal

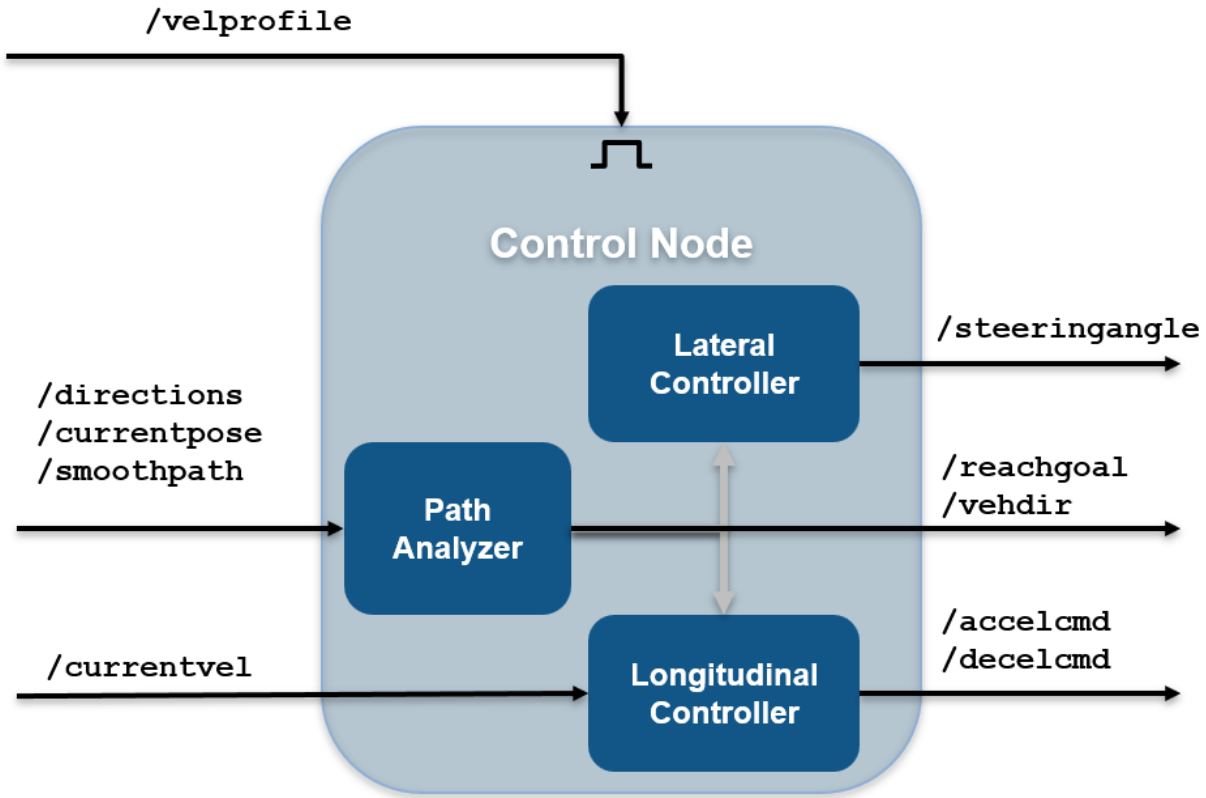
Subscribes to these topics

- /currentvel
- /currentpose
- /desiredvel
- /reachgoal

On receiving a /reachgoal message, the node runs the **exampleHelperROS2ValetPlannerCallback** callback, which plans the next segment.

Automated Parking Valet with ROS in MATLAB

- Control node:



Publishes to these topics

- /steeringangle
- /accelcmd
- /decelcmd
- /vehidir
- /reachgoal

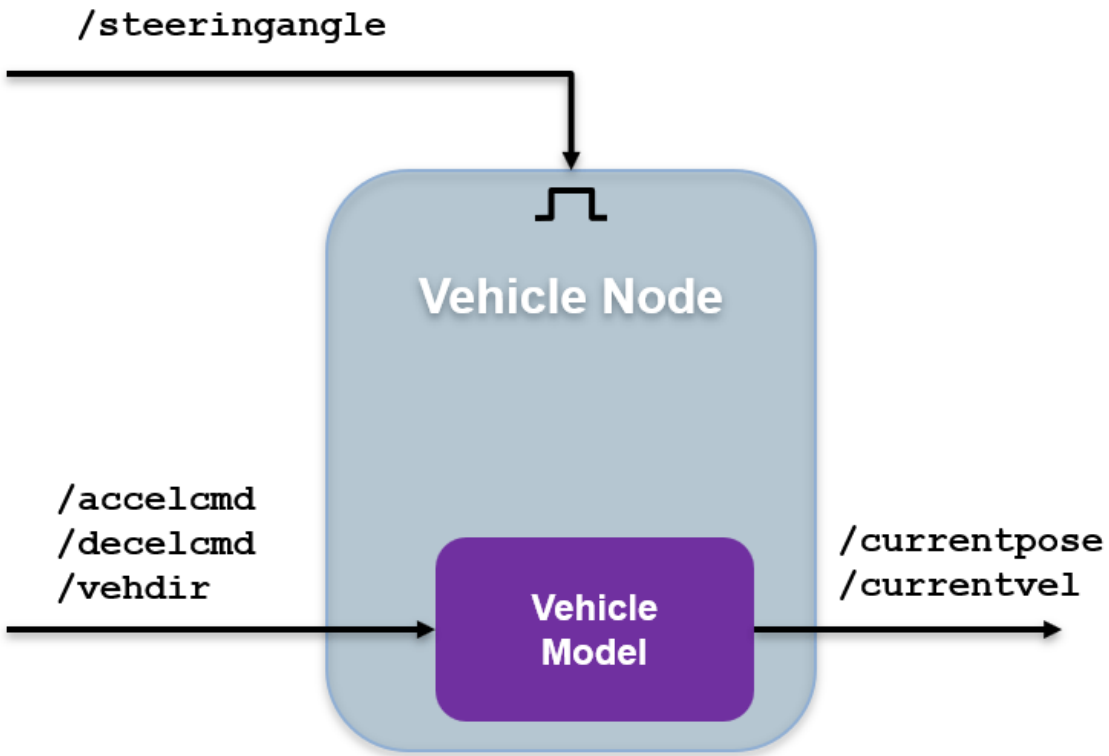
Subscribes to these topics

- /smoothpath
- /directions
- /currentvel
- /currentpose
- /desiredvel
- /nextgoal
- /velprofile

On receiving a /velprofile message, the node runs the **exampleHelperROS2ValetControlCallback** callback, which sends control messages to the vehicle

Automated Parking Valet with ROS in MATLAB

- Vehicle node:



Publishes to these topics
- /currentvel
- /currentpose

Subscribes to these topics
- /steeringangle
- /accelcmd
- /decelcmd
- /vehidir

On receiving a /steeringangle message, the vehicle simulator is run in the callback function, **exampleHelperROSValetVehicleCallback**.

ROS More advanced concepts

- Other concepts that are useful to ROS are
  - Transformation trees (TF),
  - URDF to define the robot for a simulation environment,
  - ROS bags, to record and replay data related to the robot
  - Simulation environment → Gazebo
  - Visualization interface → rviz

ROS More advanced concepts

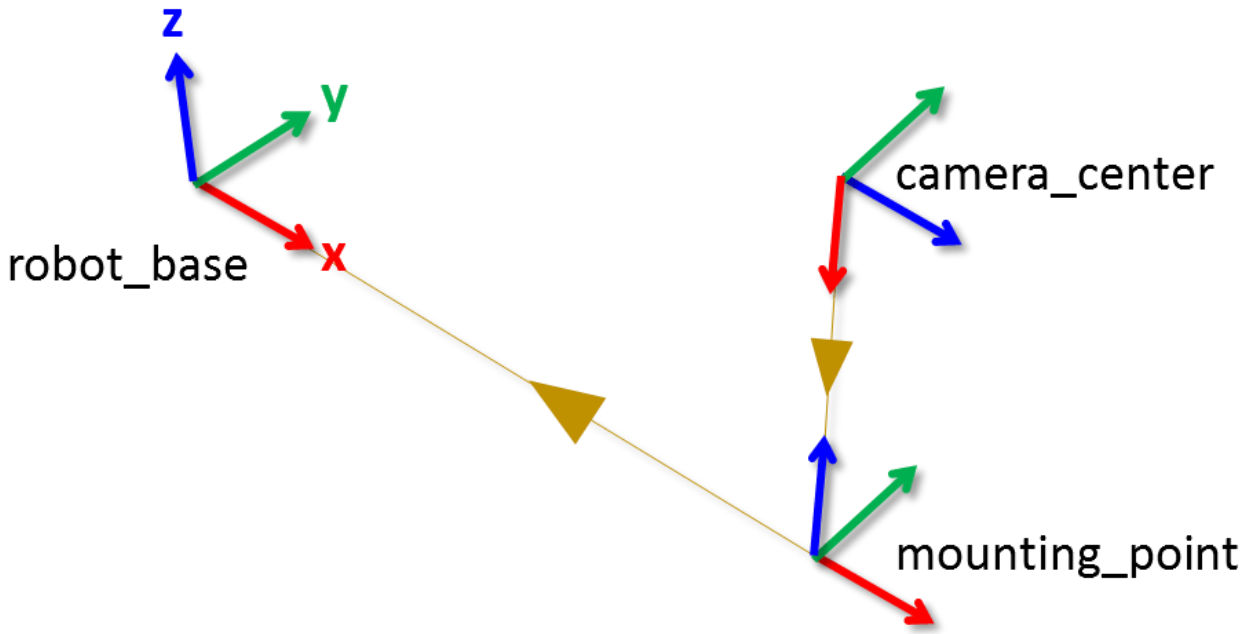
- To continue with the tutorials, please download and extract the L2_ROS_MATLAB.zip file from the Portale della Didattica and at the folder to the MATLAB path (with folders and subfolders)
  - It contains all the necessary functions to run the next tutorials
  - Make sure to have the Virtual Machine provided by MATLAB installed (the one indicated in the working setup document)

ROS TF: transformation trees

- In ROS, coordinate frames are made available to the system using tf
- Tf package provides the tools and nodes for keeping track of coordinates frames over time
- Transform are broadcasted using the topic /tf
- Understanding TF allow to extract/modify the transformations for a robot component frame (camera, LIDAR, etc) with respect to another reference frame of a robot component (wheels, base, links)

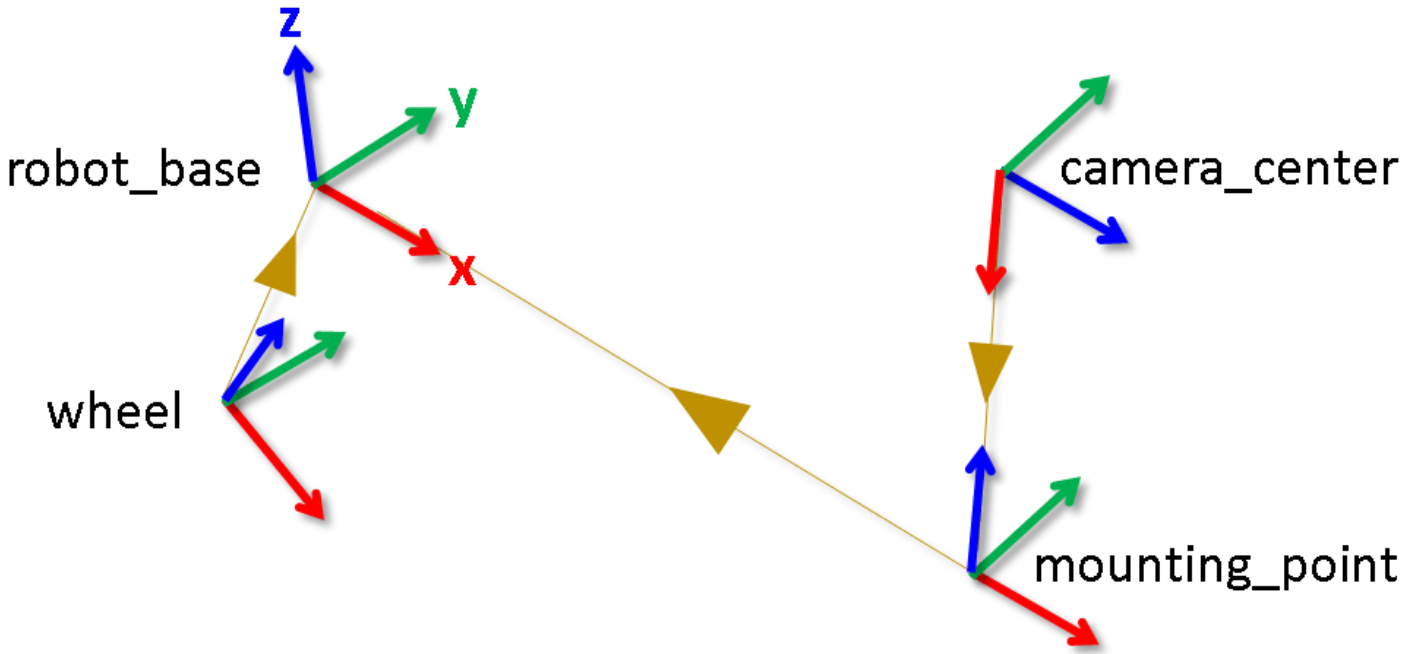
ROS TF: Tutorial in MATLAB

- In MATLAB we will use the “exampleHelperROSStartTfPublisher” to make available fake transformations, containing three coordinate frames with the transformations, and then explore them



ROS TF: Tutorial in MATLAB

- In the example it is possible to explore the different transformation coordinates, as well as adding a new one, for example, the one corresponding to a robot's wheel



ROS URDF file

- URDF (Unified Robot Description Format) is an XML file that describes the kinematic chain of a robot. It is used to model mobile robots, robotic manipulators, complex robots

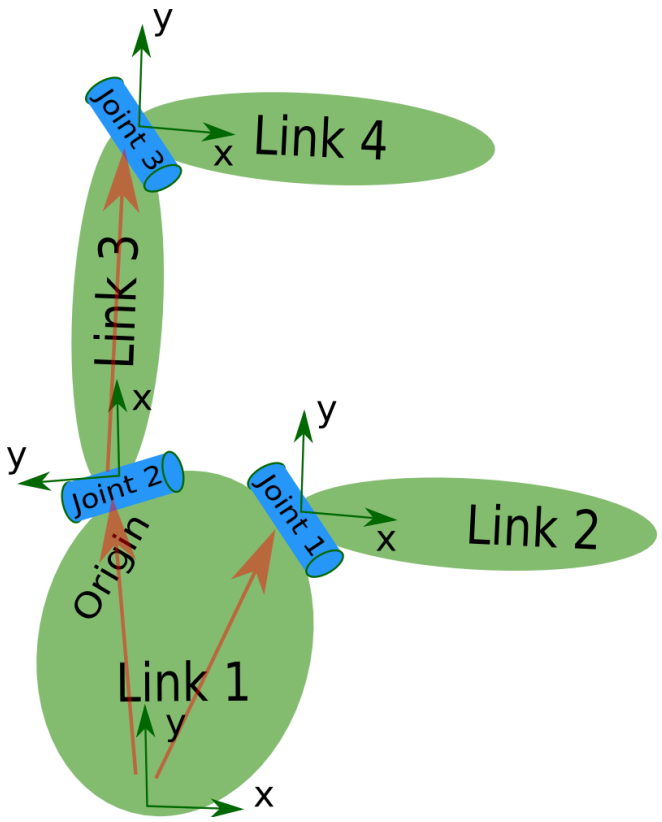
- A URDF file contains tags that hierarchically describe a robot with the links and joints composing it

ROS URDF file

Main URDF tag:

- <link> tag: describes a robot link as a rigid body

- <joint> tag: describes a robot joint and is used to connect two links called:
  - Parent link
  - Child link



ROS URDF file

- <link> tag: can contain
  - Visual tag: to define the visual aspect of the link;
  - Inertial tag: to specify dynamic features of the link;
  - Collision tag: to define the shape of the object for collision computation;

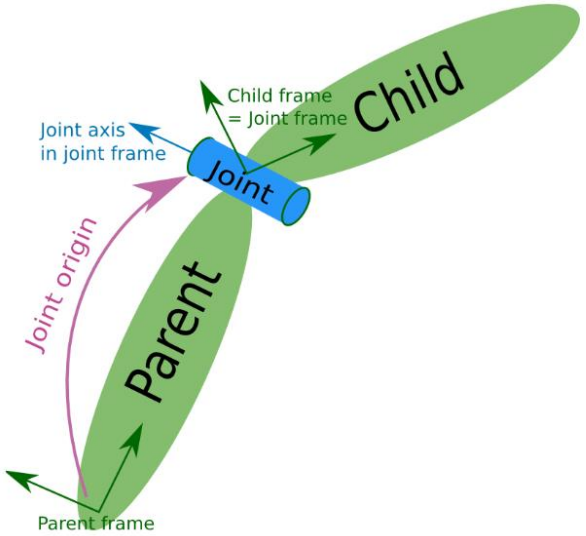
```
<link name="link_name">
    <inertial>
        <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0"/>
        <mass value="0.0"/>
        <inertia ixx="0.0" ixy="0.0" ixz="0.0" iyy="0.0" iyz="0.0" izz="0.0"/>
    </inertial>
    <visual name="">
        <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0"/>
        <geometry>
            <box size="0.0 0.0 0.0"/>
        </geometry>
        <material name="">
            <color rgba="1.0 0.0 0.0 1.0"/>
            <texture filename=""/>
        </material>
    </visual>
    <collision>
        <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0"/>
        <geometry>
            <box size="0.0 0.0 0.0"/>
        </geometry>
    </collision>
</link>
```

ROS URDF file

- <joint> tag: Kinematics and dynamics properties can be defined along with the joint type and the joint limits expressed in radians

```
<joint name="joint_name" type="revolute">
    <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0"/>
    <parent link="parent_link"/>
    <child link="child_link"/>
    <axis xyz="0.0 0.0 0.0"/>
    <limit lower="0.0" upper="0.0" effort="0.0" velocity="0.0"/>
</joint>
```

- The origin of a link is located at the origin of the joint connecting it to the parent link → the <origin> tag within the <link> tag is referring to the visual origin

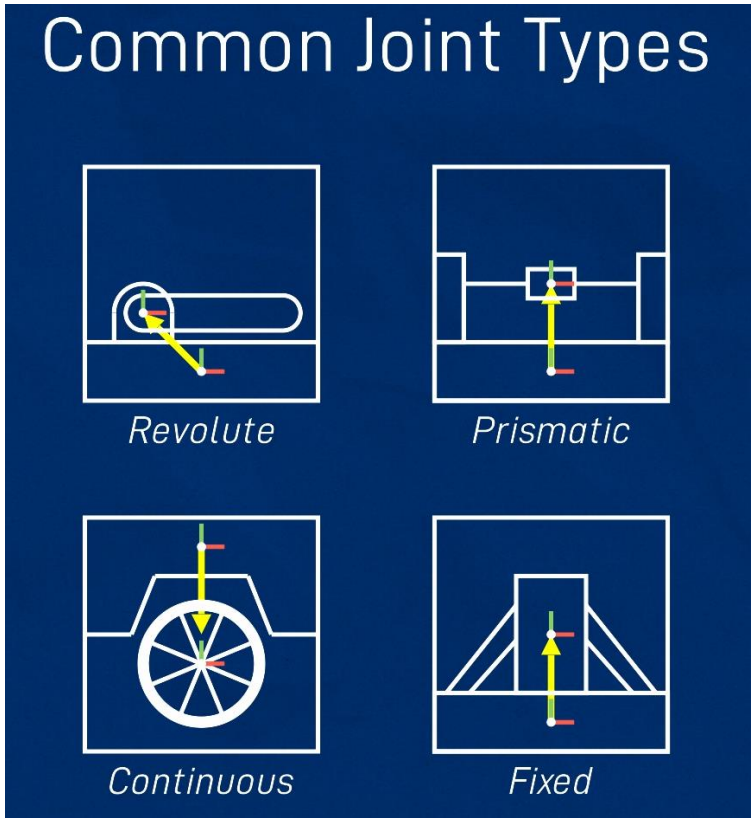


More details: https://wiki.ros.org/urdf/XML/joint

ROS URDF file

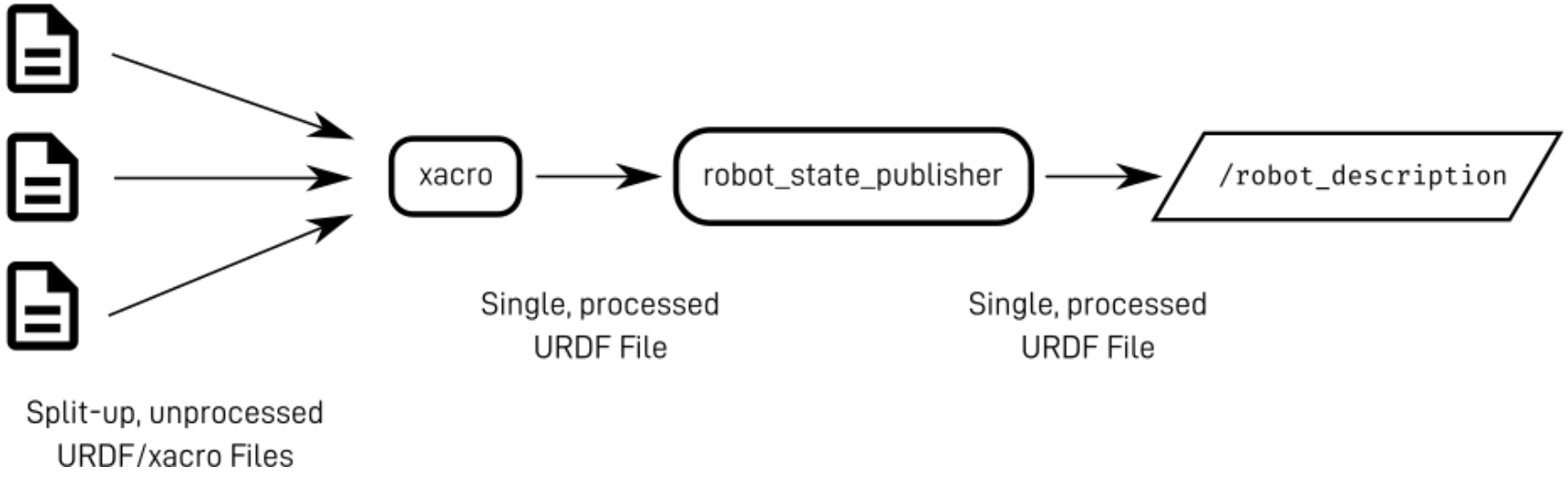
Type of joints:

- Revolute: A rotational motion, with minimum/maximum angle limits
- Continuous: A rotational motion with no limit (e.g., a wheel)
- Prismatic: A linear sliding motion, with minimum/maximum position limits
- Fixed: The child link is rigidly connected to the parent link



ROS URDF file

- To clean up an URDF you can use XACRO (XML Macro), a macro language
- You can find the .urdf or .xacro of the robot in the folder related to Robot Description in a GitHub repository



ROS URDF file

- URDF vs XACRO example:

rover.urdf

```
<robot name="my_robot">

  <!-- Links definition -->
  <link name="base_link">
    <visual>
      <origin xyz="0 0 0" />
      <geometry>
        <box size="0.3 0.2 0.2" />
      </geometry>
    </visual>
  </link>

  <link name="end_link">
    <visual>
      <origin xyz="0.5 0 0" />
      <geometry>
        <box size="0.1 0.1 0.1" />
      </geometry>
    </visual>
  </link>

  <!-- joint definition-->
  <joint name="joint1" type="revolute">
    <parent link="base_link" />
    <child link="end_link" />
    <origin xyz="0 0 0" />
    <axis xyz="0 0 1" />
    <limit lower="-3.14" upper="3.14" />
  </joint>

</robot>
```

rover.urdf.xacro

```
<robot name="my_robot">

  <!-- Definition of a single macro called "link" -->
  <xacro:macro name="link" params="name x y z height width depth">
    <link name="${name}">
      <visual>
        <origin xyz="${x} ${y} ${z}" />
        <geometry>
          <box size="${height} ${width} ${depth}" />
        </geometry>
      </visual>
    </link>
  </xacro:macro>

  <!-- Definition of a single macro called "joint" -->
  <xacro:macro name="joint" params="name type parent child">
    <joint name="${name}" type="${type}">
      <parent link="${parent}" />
      <child link="${child}" />
      <origin xyz="0 0 0" />
      <axis xyz="0 0 1" />
      <limit lower="-3.14" upper="3.14" />
    </joint>
  </xacro:macro>

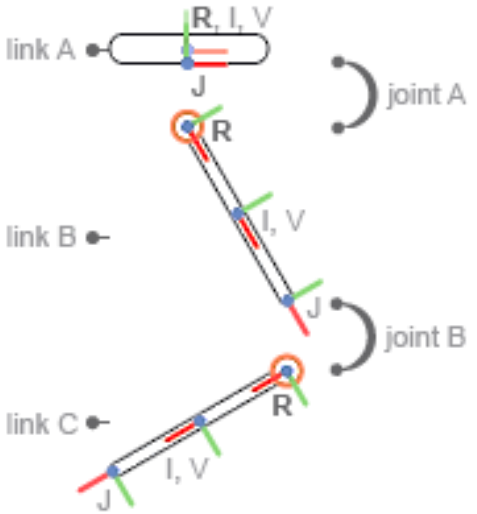
  <!-- Reuse of simple, modular and parametrized code of basic component to build a more complex structure -->
  <xacro:link name="base_link" x="0" y="0" z="0" height="0.3" width="0.2" depth="0.2"/>
  <xacro:link name="end_link" x="0.5" y="0" z="0" height="0.1" width="0.1" depth="0.1"/>
  <xacro:joint name="joint1" type="revolute" parent="base_link" child="end_link" />

</robot>
```

ROS URDF file: an example

- Let's model a double pendulum by writing a URDF file
  - Open DoublePendulum.txt file to inspect the URDF description
  - On MATLAB, navigate current folder to the one containing the DoublePendulum.txt file → rewrite the extension to .urdf
  - To import the model to MATLAB, type

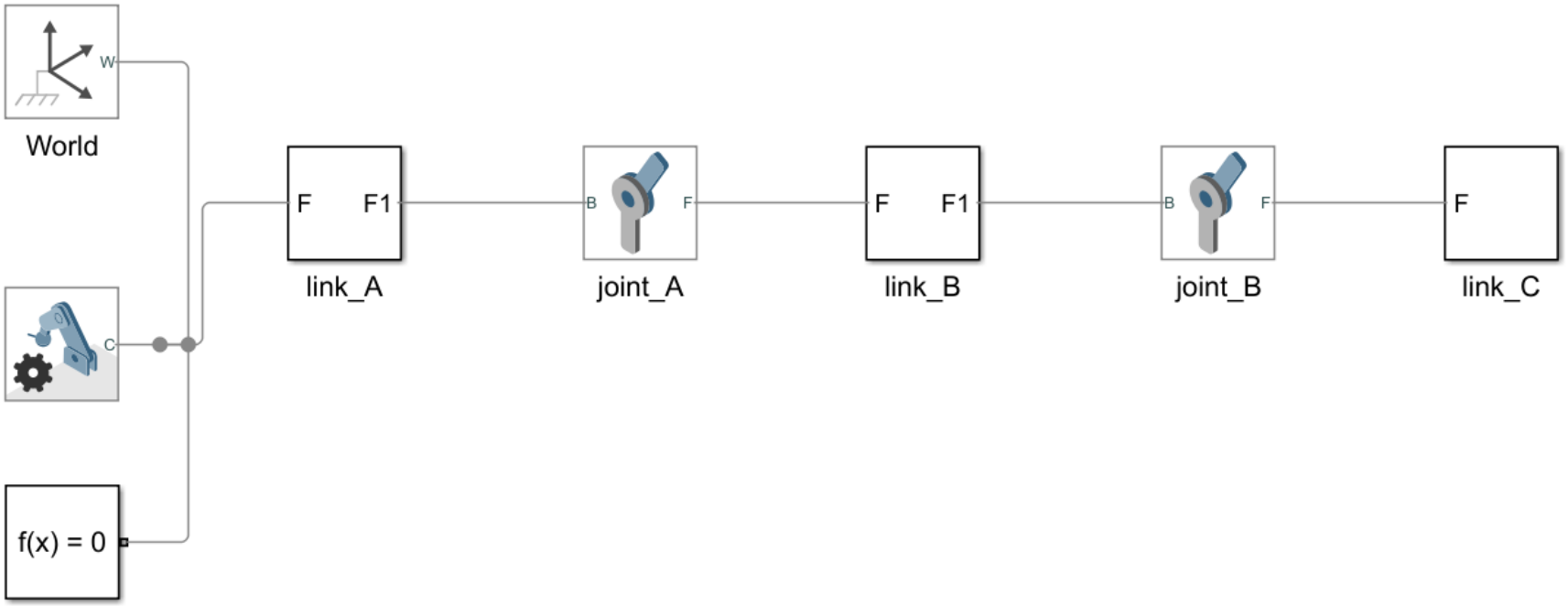
    >> smimport('DoublePendulum.urdf')





ROS URDF file: an example

- A new window will appear, with the following Block Diagram in Simulink. The file is converted into a 3D model readable in Simulink (Simscape Multibody)



ROS URDF file: an example

- In the dialog box of the joint_A block, select the **State Targets > Specify Position Target** check box and set the **Value** parameter to 30 deg. This parameter sets the starting angle of the upper joint.

- In the **Solver** pane of the **Configuration Parameters window**, click Additional options and set the Max step size parameter to 0.01. This value keeps the solver step size small enough to produce a smooth animation during simulation. Increase the value if simulation proceeds slowly.

- Update the block diagram and run the simulation. In the **Modeling tab**, click **Update Model**. You can simulate the model by selecting Run. Mechanics Explorer shows an animation of the double pendulum fall under gravity.

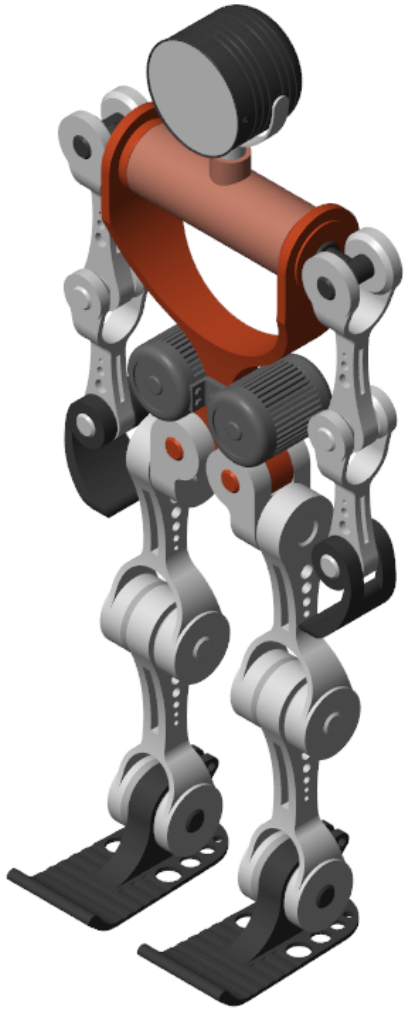
ROS URDF file: another example

- Importing a humanoid robot in MATLAB:

```
>> openExample("sm/DocImportedURDFModelExample");
>> addpath(genpath("ImportedURDFSupport"));
>> smimport("Humanoid.urdf")
```

To run a controlled example:

```
>> openExample("sm/ImportedURDFExample")
```



ROS Bag files

- The message data in ROS topics can be stored in log files called ROS bag files or *rosbag*.

- You can use MATLAB functions to access, filter and execute **offline analysis** of ROS data in these *rosbag* files. These bags are often created by subscribing to one or more ROS topics and storing the received message data in an efficient file structure.

ROS Bag files

- It is possible to explore the one of the features of *rosbag* in MATLAB, which is looking at a recorded path
- In MATLAB it is possible for example to replay the recorded path traversed by a mobile robot:

```
>> bag = rosbag('ros_turtlesim.bag');
```

- We can extract the information from a topic

```
>> bSel = select(bag,'Topic','/turtle1/pose');
```

ROS Bag files

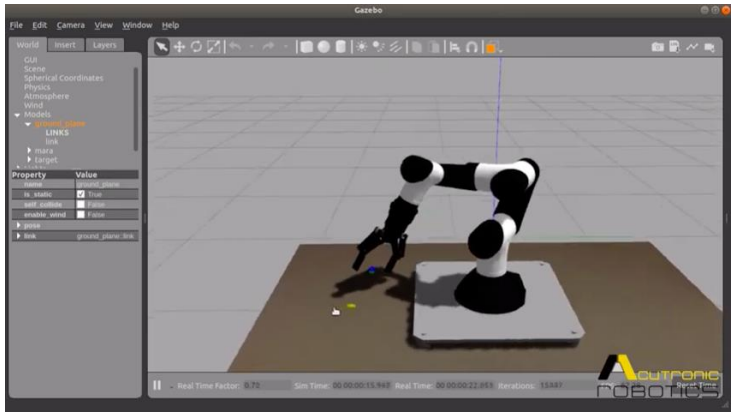
- Then we can read the messages contained in the topic and then plot it:

```
>> msgStructs = readMessages(bSel,'DataFormat','struct’);
>> msgStructs{1}

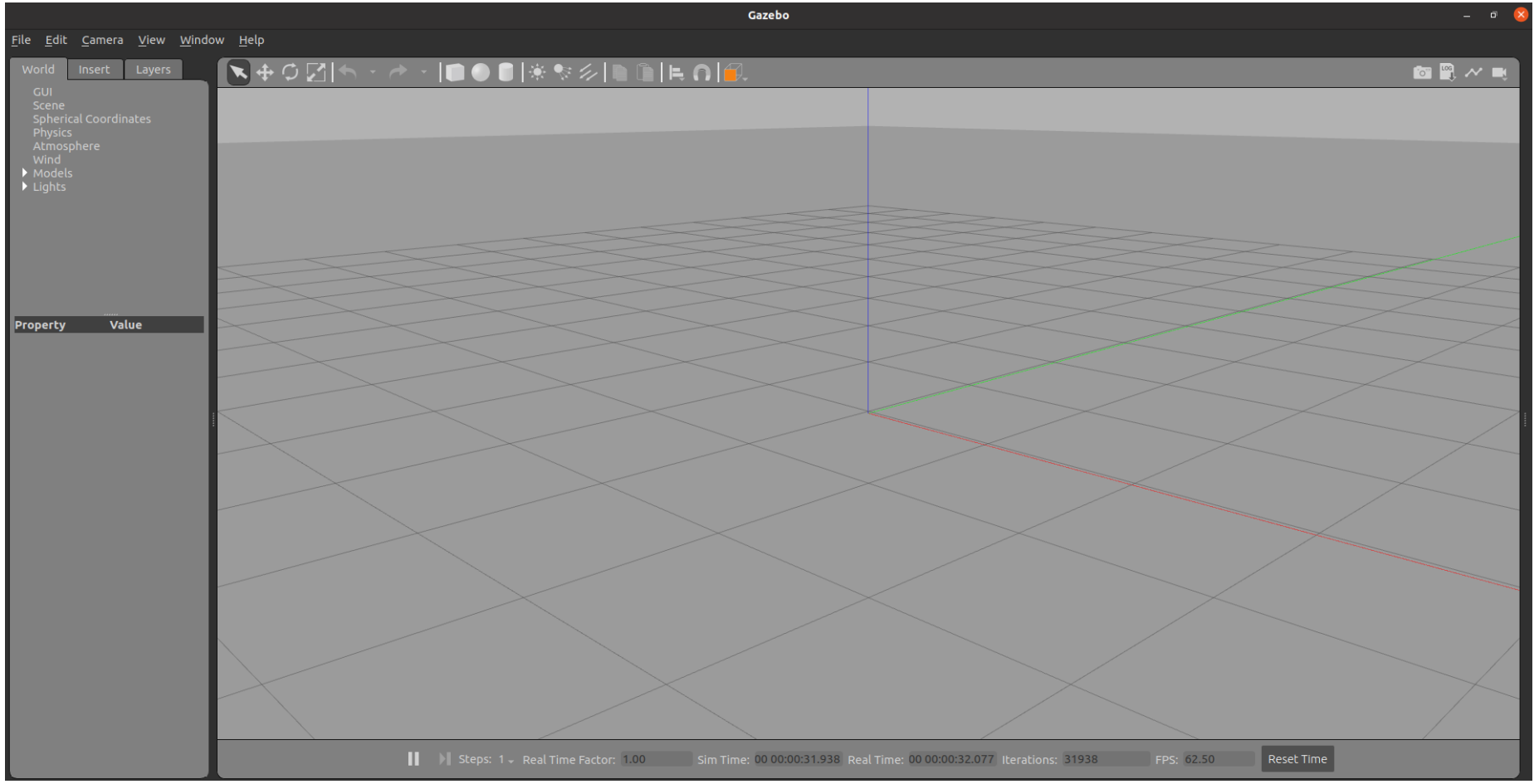

>> xPoints = cellfun(@(m) double(m.X),msgStructs);
>> yPoints = cellfun(@(m) double(m.Y),msgStructs);
>> plot(xPoints,yPoints)
```

ROS Gazebo

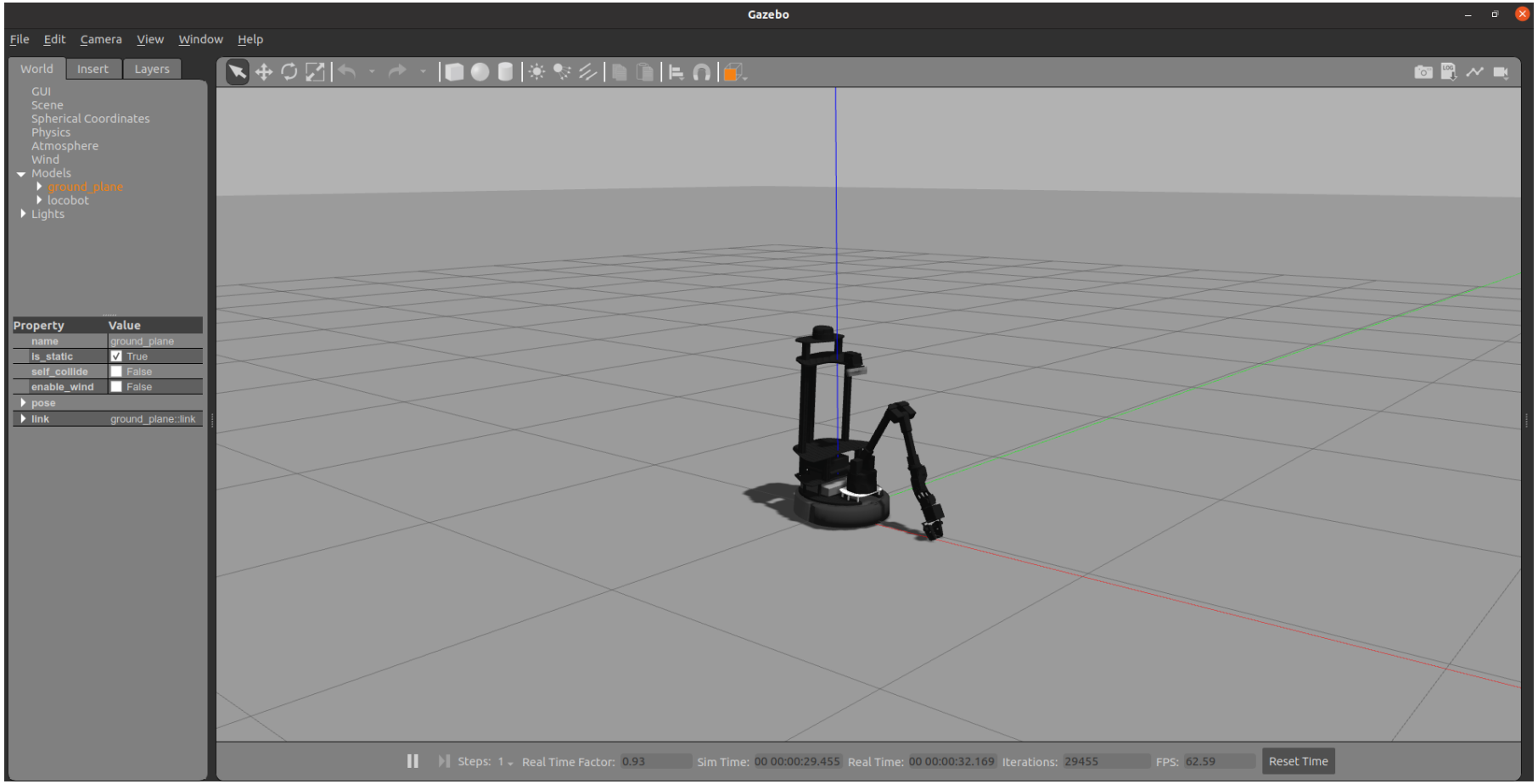
- Gazebo is an open-source 3D robot simulation tool that provides a comprehensive platform for modeling robots, sensors, and environments.
- It supports different physics engines to simulate gravity, friction, torques, and any other real-life conditions that could affect robot performance.
- It is used for robotics research, development, and prototyping.



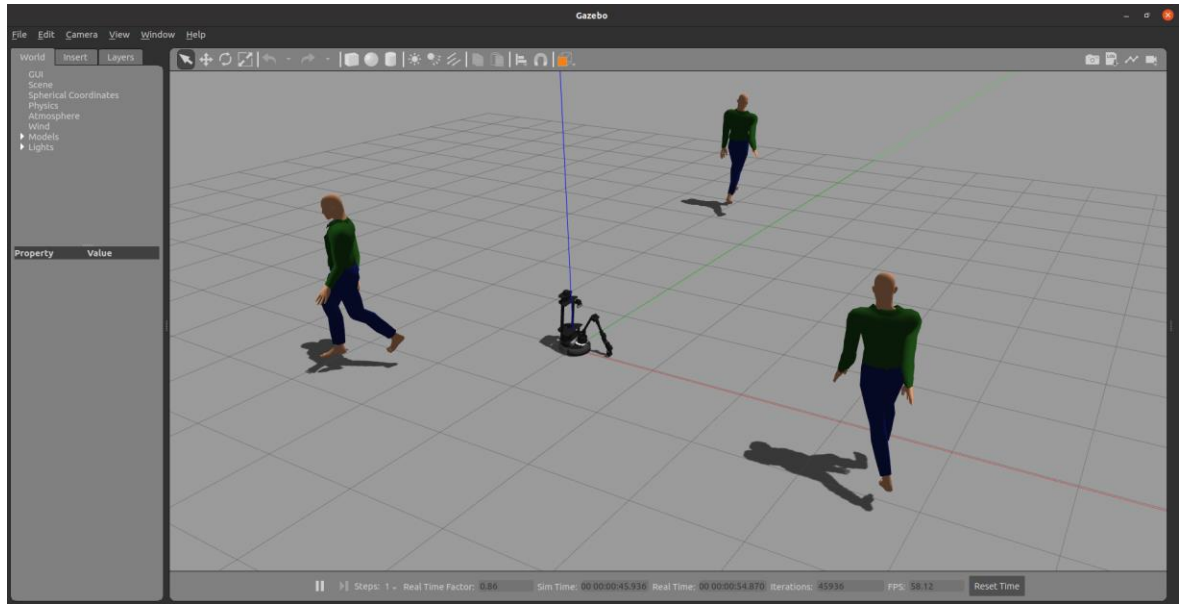
ROS Gazebo – main window

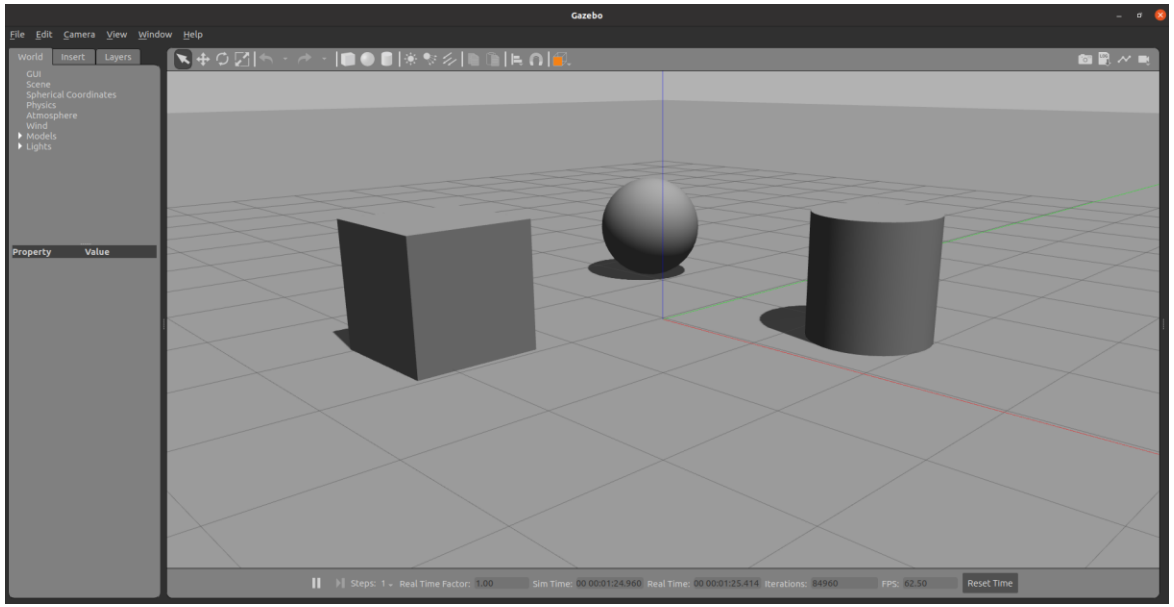


ROS Gazebo – adding URDF files



ROS Gazebo - adding obstacles

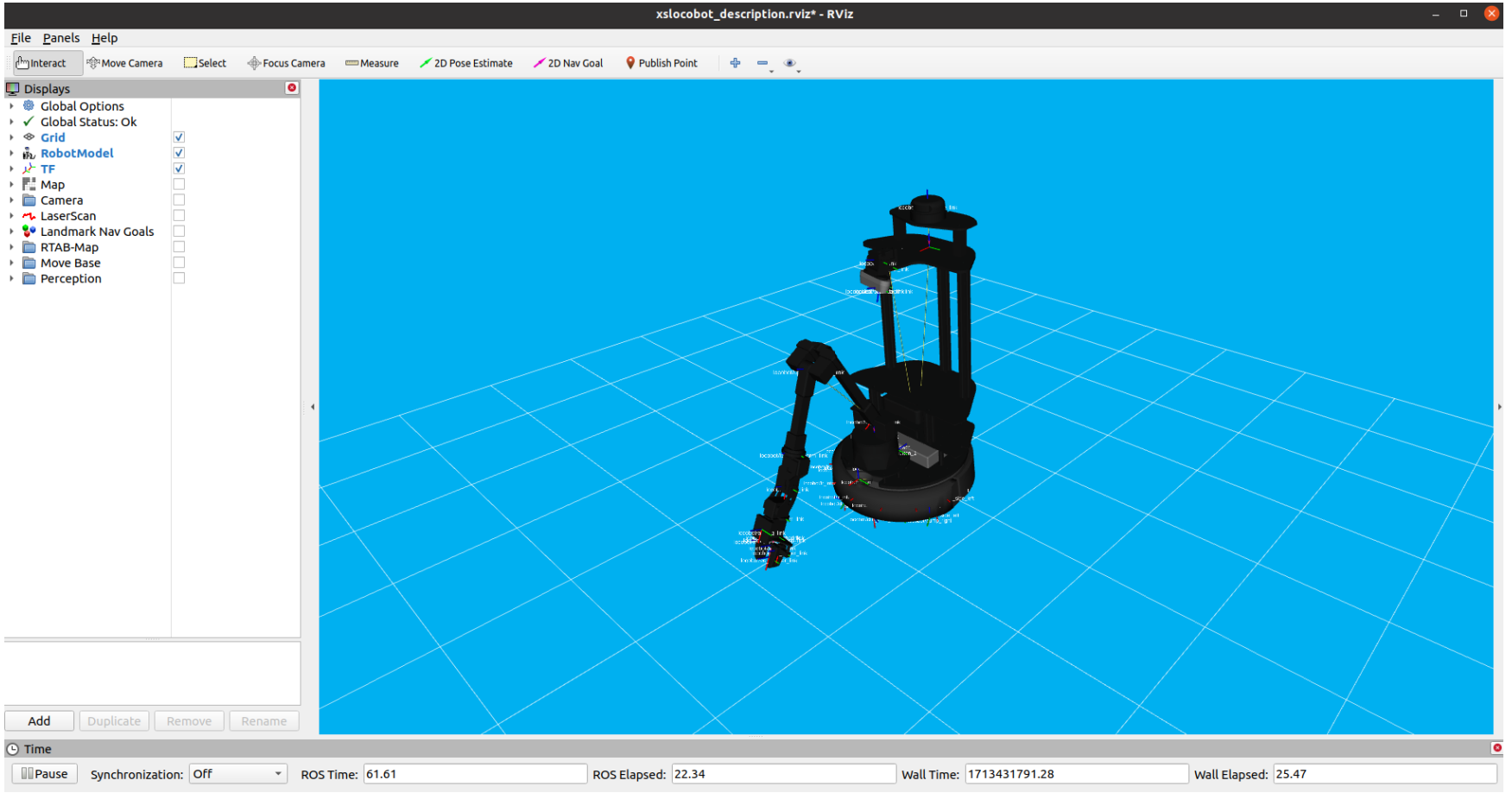




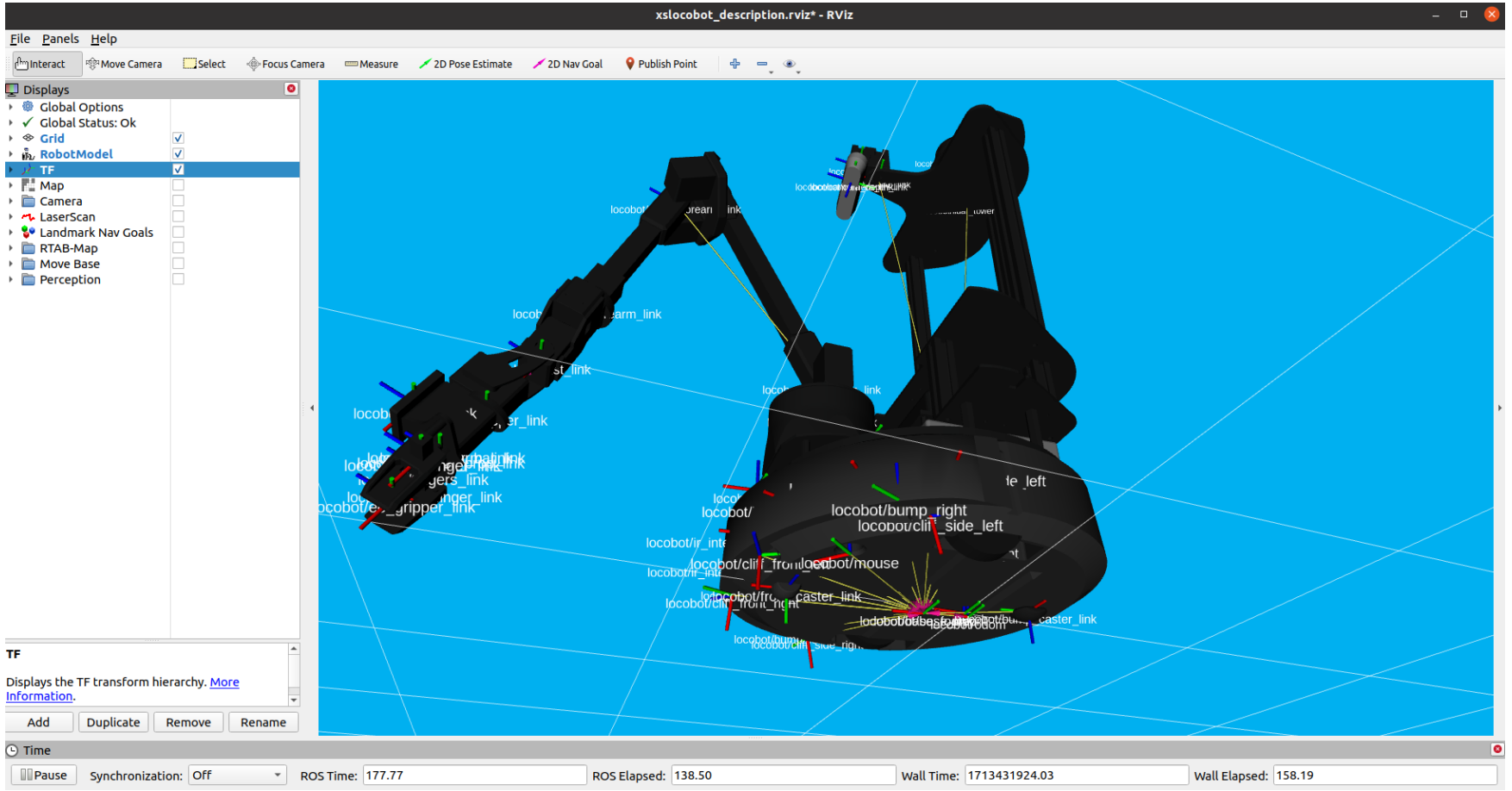
ROS Rviz: ROS 3D robot vizualizer

- It is a real-time 3D visualization tool integrated in ROS. It helps users to better understand the operation of their robotic systems by visualizing data coming from sensors and other robot components.
- It allows to display various information such as maps, robot models, sensor data (like laser scan data or camera images), planned paths, reference frames and more.
- The real-time data visualization is extremely useful during development, debugging, and testing of robotic systems, enabling developers to have a clear understanding of what the robot "sees" and "thinks" during its operation.

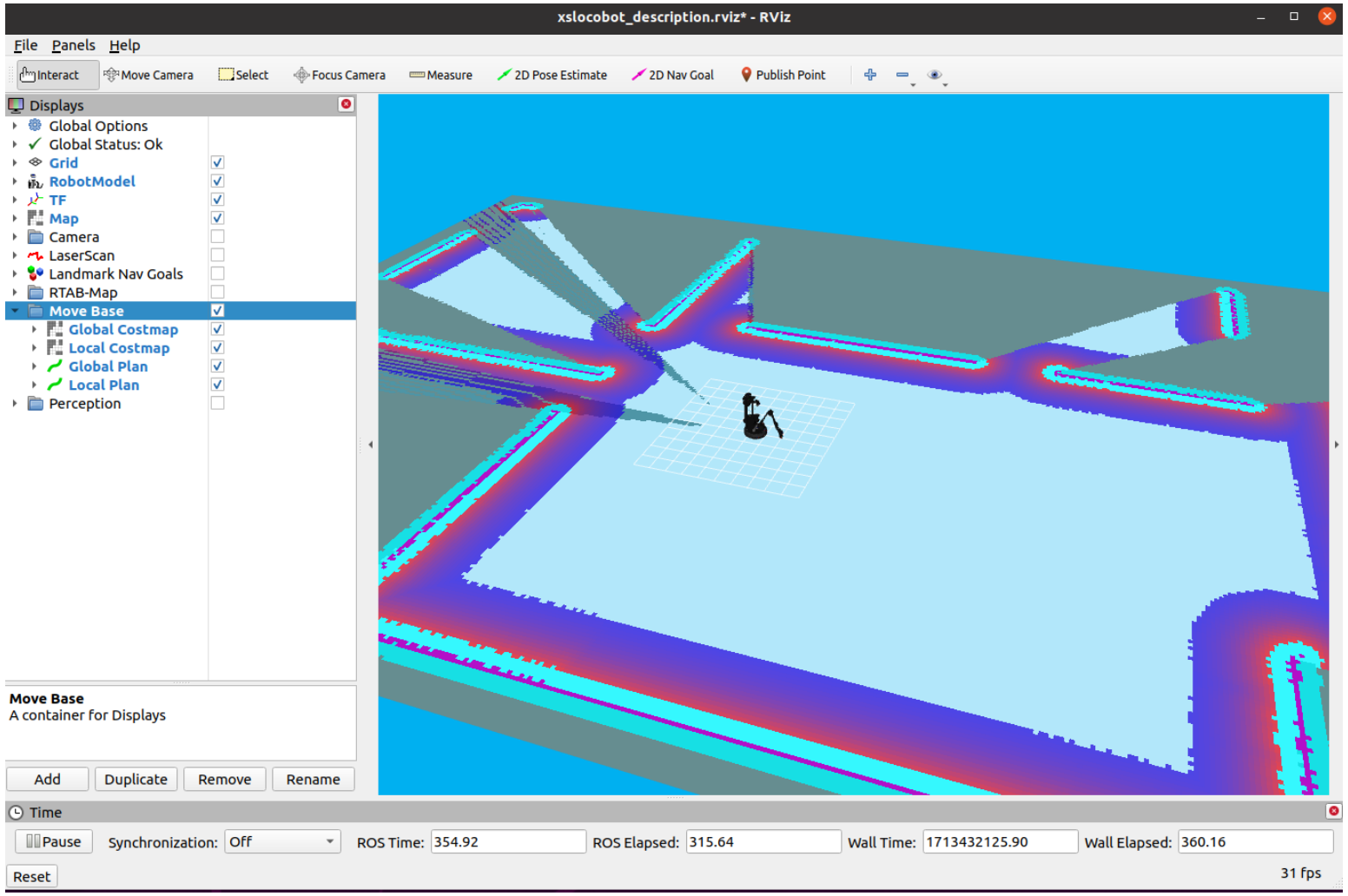
ROS Rviz – main window



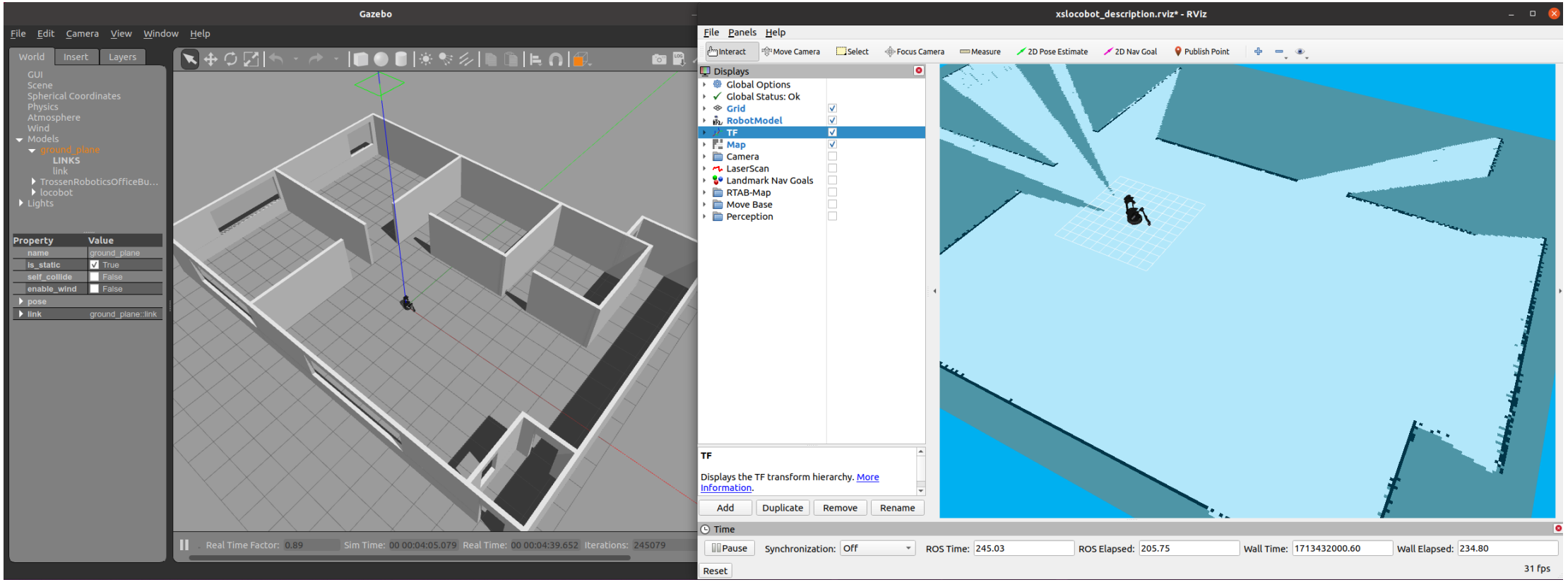
ROS Rviz – tf reference frames



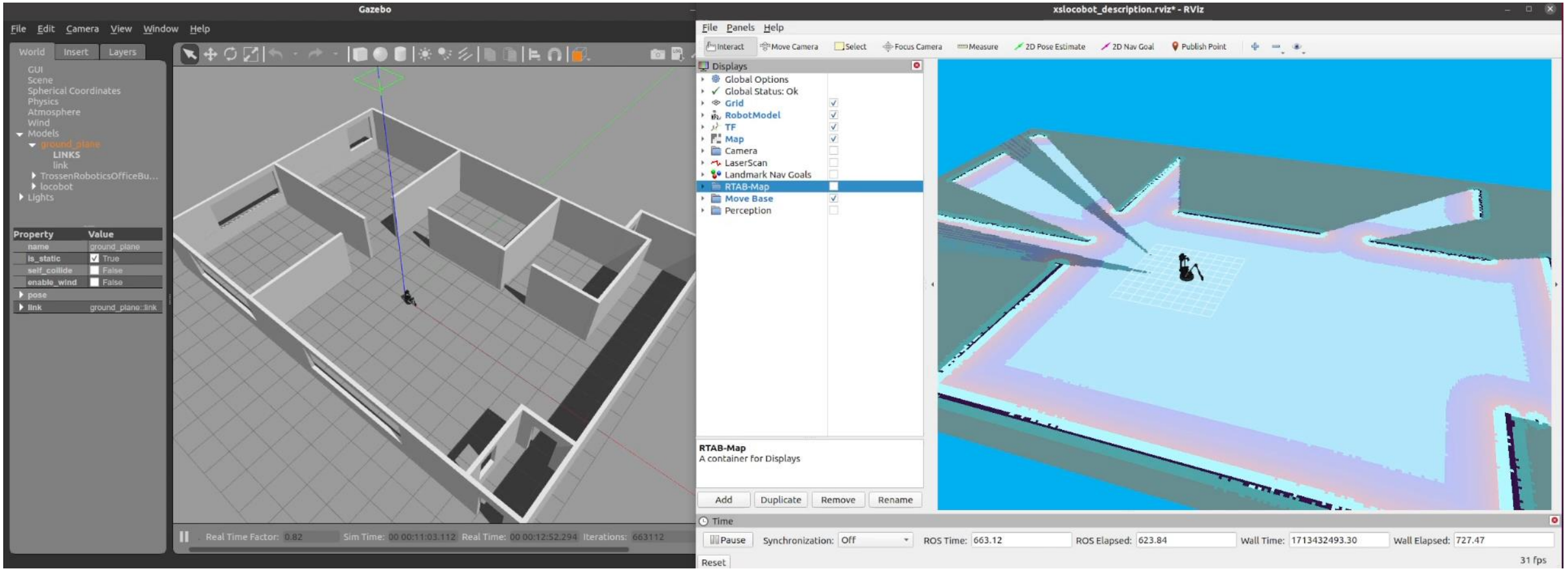
ROS Rviz – move_base data



ROS Rviz: SLAM data



ROS Rviz + Gazebo



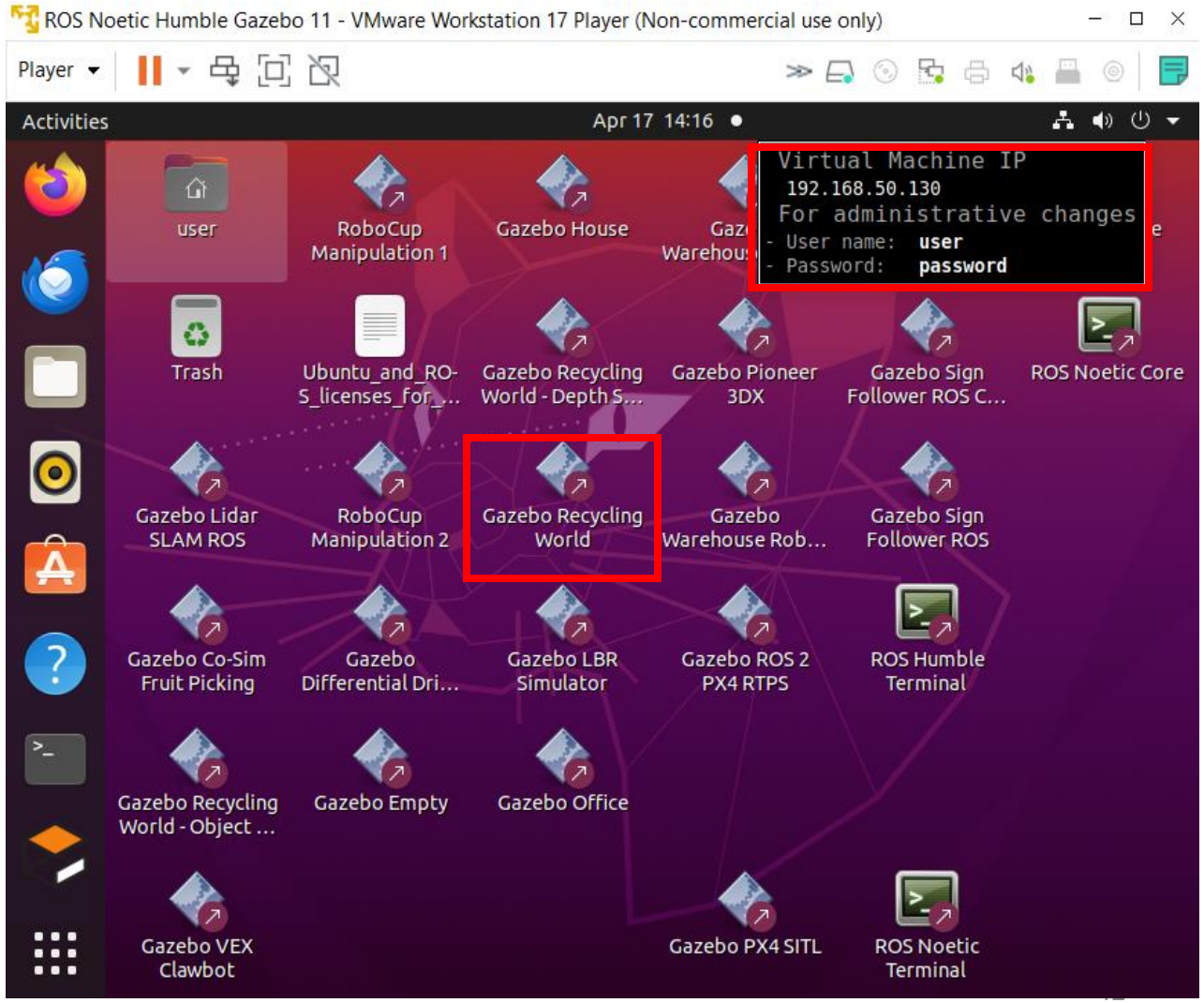
ROS Gazebo tutorial

- It is possible to interact with the Gazebo using MATLAB and a virtual machine: It allows to emulate a connection between a real robot and the system you are using to control/monitor the robot

- To open Gazebo, we can open the Virtual Machine, but first, set this:
  - In VM settings, VM → Settings → Hardware → Display, disable Accelerate 3D graphics.
- Then we can initialize the Ubuntu Virtual Machine

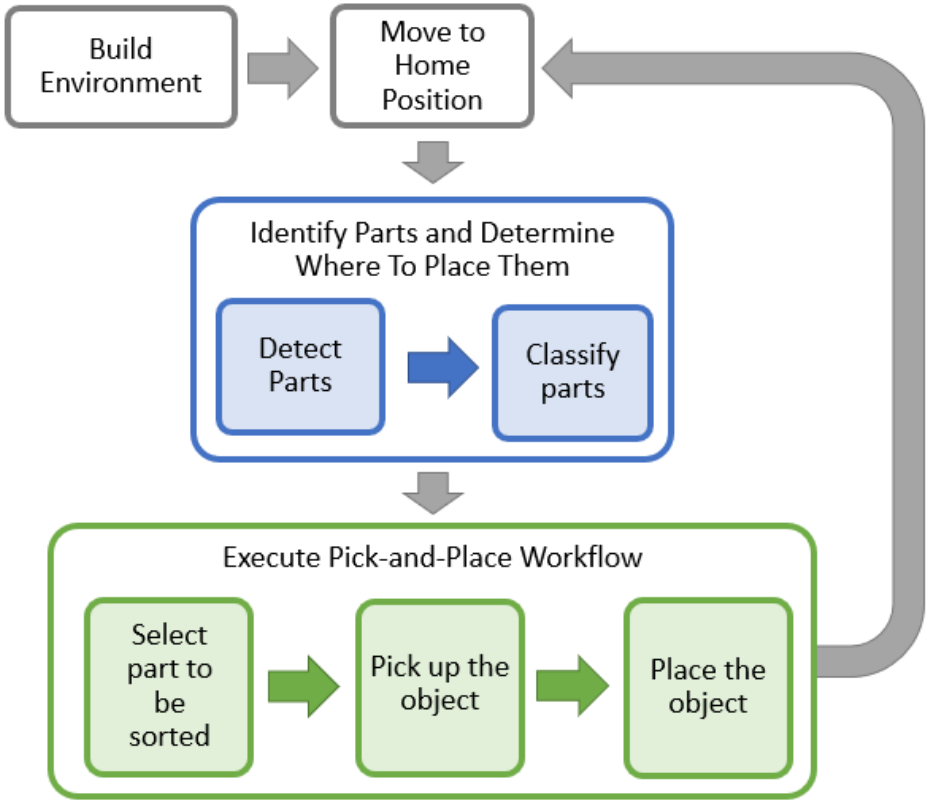
ROS Gazebo tutorial

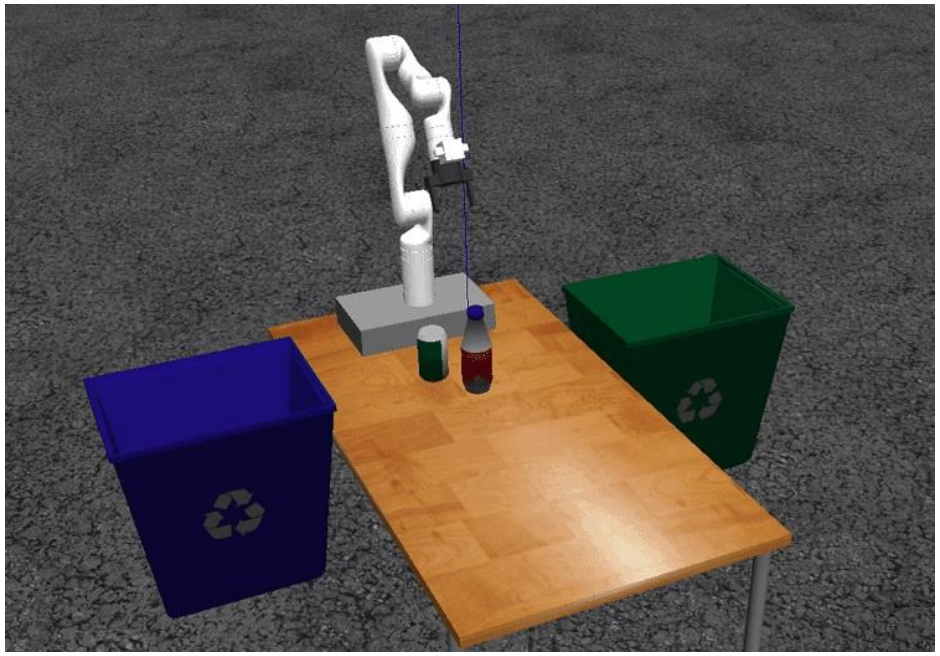
- Desktop view of the Virtual Machine. We will open «Gazebo Recycling World»
- Use the IP address provided by the Virtual Machine
  - In this example is 192.168.50.130
  - It will be the IP address to connect with the Robot in the virtual environment



ROS Gazebo tutorial

- The simulation environment opens a virtual world with a manipulator with the task of picking and placing some objects like cans and bottles

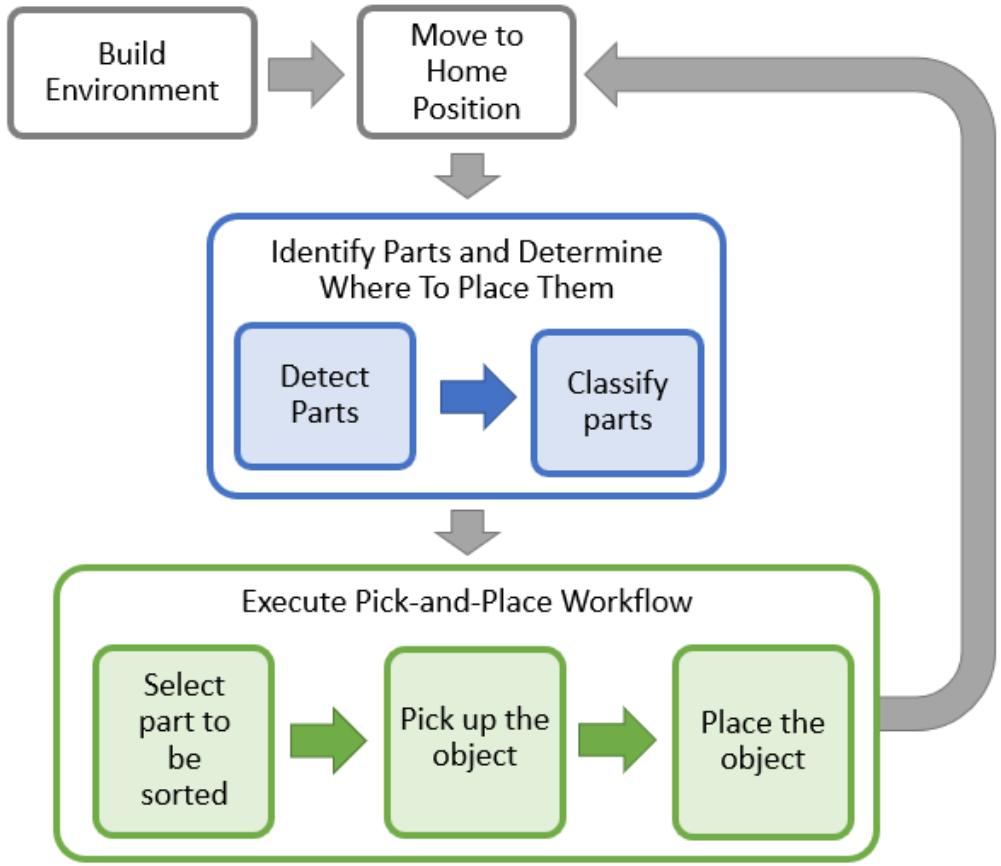




ROS Gazebo tutorial

- The robot uses ROS control to command all the joints
  - It implements a PID controller

- The object identification part is performed using Deep Learning algorithms (YOLOv2), and it process the image captured by the camera



ROS Gazebo tutorial

- The overall process is controlled using stateflow, whose diagram can be seen by typing:

>> edit exampleHelperFlowChartPickPlaceROSGazebo.sfx

Where the block functionalities are:

MoveToHome, Idle, DetectParts, ClassifyParts and PickPlaceParts

ROS Gazebo tutorial

- To connect with the Robot, we need to click PLAY in the Gazebo environment, and then initialize the ROS Master using the IP address of the VM:

>> rosinit(rosIP, 11311); → rosIP is the IP of your VM

In this way you can command the Robot in the virtual environment

ROS Gazebo tutorial

- Let’s load the data related to the robot and initialize the starting pose:

```
>> load('exampleHelperKINOVAGen3GripperROSGazebo.mat’);

>> initialRobotJConfig = [3.5797 -0.6562 -1.2507 -0.7008 0.7303 -2.0500 -1.9053];
>> endEffectorFrame = "gripper";
```

ROS Gazebo tutorial

- Let’s initialize also the coordinator handle to manage robot control, by giving the robot model, initial configuration, and end-effector name:

```
>> coordinator = exampleHelperCoordinatorPickPlaceROSGazebo(robot, initialRobotJConfig, endEffectorFrame);

>> coordinator.HomeRobotTaskConfig = getTransform(robot, initialRobotJConfig, endEffectorFrame);

>> coordinator.PlacingPose{1} = trvec2tform([[0.2 0.55 0.26]])*axang2tform([0 0 1 pi/2])*axang2tform([0 1 0 pi]);

>> coordinator.PlacingPose{2} = trvec2tform([[0.2 -0.55 0.26]])*axang2tform([0 0 1 pi/2])*axang2tform([0 1 0 pi]);

>> coordinator.FlowChart = exampleHelperFlowChartPickPlaceROSGazebo('coordinator', coordinator);
```

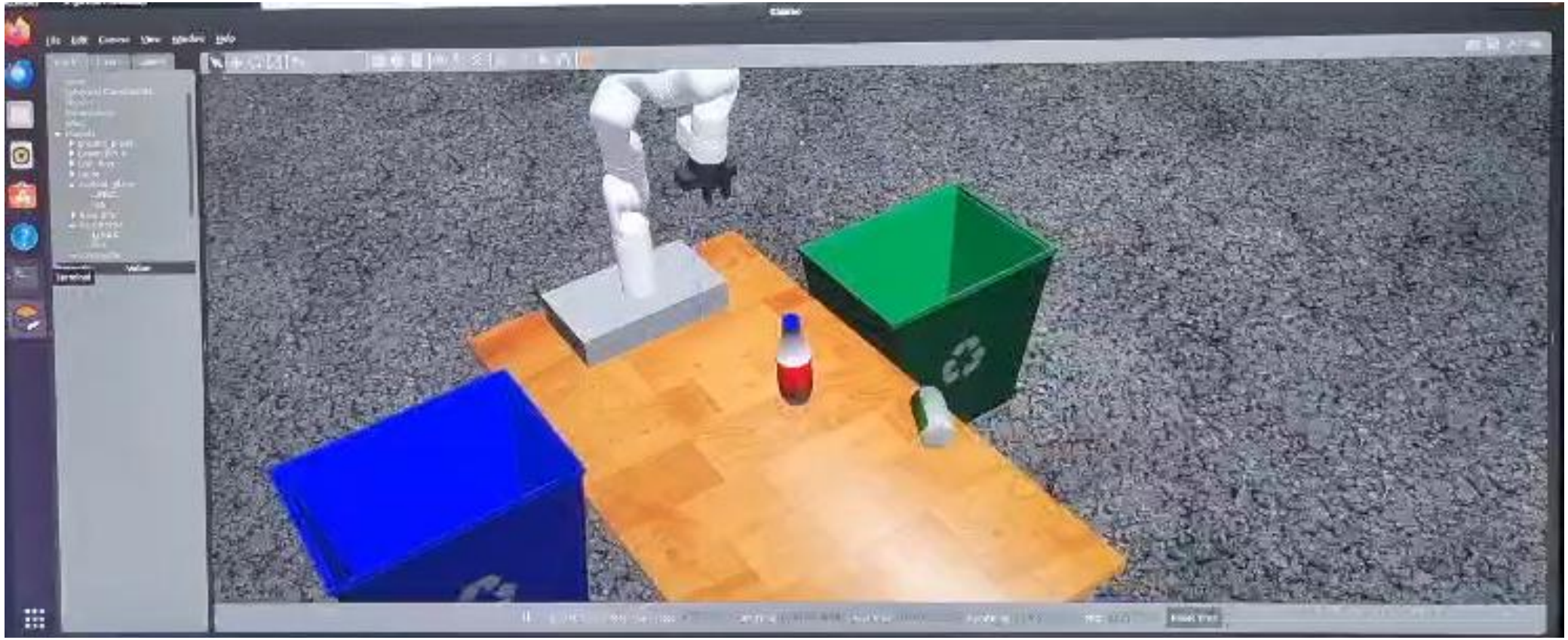
ROS Gazebo tutorial

- Finally, let’s execute the robot’s motion, but in a convenient way by defining a dialog to start the execution:

```
answer = questdlg('Do you want to start the pick-and-place job now?', ...
      'Start job','Yes','No', 'No');

switch answer
   case 'Yes'
      % Trigger event to start Pick and Place in the Stateflow Chart
      coordinator.FlowChart.startPickPlace;
   case 'No'
      coordinator.FlowChart.endPickPlace;
      delete(coordinator.FlowChart)
      delete(coordinator);
end
```

ROS Gazebo tutorial



ROS Tutorial links

- Automated Parking Valet with ROS in MATLAB
- ROS tf
- URDF
- ROS bag
- Gazebo: Pick and Place with a KinovaGen3